

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Высшая школа программной инженерии

Работа допущена к защите

Директор ВШПИ

_____ П.Д.Дробинцев

« ____ » _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

работа бакалавра

РАЗРАБОТКА АВТОМАТИЗИРОВАННОЙ СИСТЕМЫ КОНТРОЛЯ И УЧЕТА РАБОЧЕГО ВРЕМЕНИ СОТРУДНИКОВ

по направлению подготовки (специальности)

09.03.04 Программная инженерия

Направленность (профиль)

**09.03.04_01 Технология разработки и сопровождения качественного
программного продукта**

Выполнил студент гр.

5130904/10101

К.В.Козлов

Руководитель

доцент

П.Д.Дробинцев

Консультант

по нормоконтролю

Е.Г.Локшина

Санкт-Петербург
2025

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

УТВЕРЖДАЮ
Директор ВШПИ
П. Д. Дробинцев
«__» _____ 2025 г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы

студенту Козлову Кириллу Владимировичу, 5130904/10101
фамилия, имя, отчество (при наличии), номер группы

1. Тема работы: Разработка автоматизированной системы контроля и учета рабочего времени сотрудников
2. Срок сдачи студентом законченной работы: 17.05.2025
3. Исходные данные по работе: Документация на языки Python, JavaScript, документация СУБД PostgreSQL
4. Содержание работы (перечень подлежащих разработке вопросов): 1) Введение. 2) Обзор предметной области, содержащий описание существующих решений и их сравнительный анализ. 3) Архитектура предлагаемого решения. 4) Реализация предлагаемого решения. 5) Результаты. 6) Выводы.
5. Перечень графического материала (с указанием обязательных чертежей):

6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии): Vue.js, FastAPI, PostgreSQL, RESTful API, Visual Studio Code, Git, Uvicorn.
7. Консультанты по работе: отсутствуют.

8. Дата выдачи задания: 14.04.2025

Руководитель ВКР _____ П.Д. Дробинцев

(подпись)

инициалы, фамилия

Задание принял к исполнению 14.04.2025

Студент _____ К.В. Козлов

(подпись)

инициалы, фамилия

РЕФЕРАТ

На 57 с., 16 рисунков, 4 таблицы

КЛЮЧЕВЫЕ СЛОВА: АВТОМАТИЗАЦИЯ, УЧЕТ РАБОЧЕГО ВРЕМЕНИ, БЕЗОПАСНОСТЬ ДАННЫХ, VUE.JS, API, ASTRA LINUX.

Тема выпускной квалификационной работы: «Разработка автоматизированной системы контроля и учета рабочего времени сотрудников».

Данная работа посвящена созданию комплексной автоматизированной системы контроля и учета рабочего времени сотрудников, направленной на повышение эффективности управления персоналом и оптимизацию бизнес-процессов. Особое внимание уделено требованиям к безопасности и локализации данных, что актуально для государственных предприятий, использующих отечественные операционные системы, такие как Astra Linux.

В рамках работы проведен анализ существующих решений на рынке, выявлены их преимущества и недостатки. Это позволило сформулировать требования к новой системе, сочетающей высокую безопасность, масштабируемость и удобство использования [6]. Архитектура системы построена на микросервисном подходе, обеспечивающем гибкость и независимое обновление компонентов. Для реализации клиентской части был выбран фреймворк Vue.js с использованием LocalStorage для временного хранения данных и синхронизации с сервером. Серверная часть реализована с применением фреймворка FastAPI, обеспечивающим быструю обработку запросов, и реляционной системы управления базами данных PostgreSQL для хранения данных [2].

Результаты работы оформлены в виде полнофункционального программного продукта с подробной технической документацией. Разработка успешно апробирована на предприятии военно-промышленного комплекса, где получила положительные отзывы специалистов по управлению персоналом. Основные выводы были представлены на внутреннем семинаре, что подтверждает практическую ценность предложенного решения.

ABSTRACT

57 pages, 16 figures, 4 tables

KEYWORDS: AUTONATION, WORK TIME TRACKER, DATA SECURITY, VUE.JS, API, ASTRA LINUX.

Title of the Graduate Qualification Work: "Development of an Automated System for Employee Work Time Control and Accounting."

This work is dedicated to the creation of a comprehensive automated system for controlling and accounting employee work time, aimed at improving personnel management efficiency and optimizing business processes. Special attention is given to data security and localization requirements, which are particularly relevant for government enterprises using domestic operating systems such as Astra Linux.

The study includes an analysis of existing market solutions, identifying their strengths and weaknesses. This analysis enabled the formulation of requirements for a new system that combines high security, scalability, and ease of use. The system architecture is based on a microservice approach, providing flexibility and independent component updates. For the client side, the Vue.js framework was chosen, utilizing LocalStorage for temporary data storage and synchronization with the server. The server side is implemented using the FastAPI framework, which ensures fast request processing, alongside the PostgreSQL relational database management system for data storage.

The results of the work are presented as a fully functional software product accompanied by detailed technical documentation. The development has been successfully tested at a defense industry enterprise, where it received positive feedback from personnel management specialists. The main conclusions were presented at an internal seminar, confirming the practical value of the proposed solution.

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И ОПРЕДЕЛЕНИЙ	8
ВВЕДЕНИЕ.....	10
ГЛАВА 1. ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ.....	13
1.1. Общие принципы построения систем контроля рабочего времени	13
1.1.1. Видеонаблюдение	13
1.1.2. Биометрия	14
1.1.3. Электронные карты.....	15
1.1.4. Программные системы	16
1.2. Анализ существующих решений.....	16
1.2.1. Yaware.TimeTracker	17
1.2.2. StaffCop	17
1.2.3. Мегаплан.....	18
1.2.4. Kickidler.....	19
1.2.5. CrocoTime	19
1.3. Выводы.....	21
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ РЕШЕНИЯ	23
2.1. Требования к разрабатываемому решению.....	23
2.1.1. Функциональные требования	23
2.1.2. Требования к безопасности.....	25
2.1.3. Требования к надежности	26
2.1.4. Требования к составу и параметрам технических средств.....	27
2.2. Обоснование выбора подхода к разработке	28
2.2.1. Микросервисный подход	28
2.2.2. Монолитный подход.....	30
2.3. Обоснование архитектурных решений.....	31
2.3.1. Клиентская часть.....	31
2.3.2. Серверная часть.....	32
2.3.3. Сетевые технологии.....	33
2.3.4. Дополнительные инструменты.....	34
ГЛАВА 3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ.....	35

3.1. Описание функциональности	35
3.1.1. Система аутентификации и авторизации	36
3.1.2. Модуль учета рабочего времени	37
3.1.3. Система мониторинга активности	38
3.1.4. Модуль отчетности и аналитики	40
3.1.5. Модуль администрирования системы	42
3.1.6. Управление учетной записью	44
3.2. Обеспечение требований к безопасности.....	45
3.3. Обеспечение требований к надежности	46
ГЛАВА 4. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ И ОЦЕНКА ЭФФЕКТИВНОСТИ	49
4.1. Тестирование приложения	49
4.2. Результаты проверок на тестовом стенде.....	49
4.2.1. Разворачивание приложения	50
4.2.2. Работа с учетными записями	50
4.2.3. Генерация отчетов	51
4.3. Оценка эффективности системы	52
ЗАКЛЮЧЕНИЕ	54
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	56

СПИСОК СОКРАЩЕНИЙ И ОПРЕДЕЛЕНИЙ

Astra Linux – это российский дистрибутив операционной системы Linux, ориентированный на обеспечение повышенной безопасности.

Data Loss Prevention – это комплекс технологий и процессов, направленных на предотвращение утечки, потери или несанкционированного доступа к конфиденциальным данным.

Graceful degradation – это способность системы сохранять основные функции и обеспечивать минимально необходимый уровень работы при частичной недоступности или сбоях отдельных компонентов.

JavaScript – высокоуровневый язык программирования, используемый преимущественно для создания интерактивных элементов на веб-страницах и разработки клиентской части веб-приложений.

Аутентификация – это процесс проверки подлинности пользователя или устройства, подтверждающий их идентичность для предоставления доступа к системе.

Валидация – это процесс проверки корректности и соответствия входных данных заданным требованиям перед их обработкой или сохранением.

Веб-приложение – это программное приложение, доступное через веб-браузер и работающее на сервере, обеспечивающее взаимодействие пользователя с функционалом через интернет.

Дистрибутив – это комплект программного обеспечения, объединённый в готовый к установке и использованию пакет, например, операционная система с набором приложений.

Микросервис – это автономный программный компонент, реализующий ограниченную бизнес-функцию.

ОС (операционная система) – это системное программное обеспечение, управляющее аппаратными ресурсами компьютера и обеспечивающее платформу для запуска приложений.

ПО (программное обеспечение) – это набор программ и инструментов, которые позволяют компьютеру или другому вычислительному устройству выполнять различные задачи.

Стек – это набор библиотек, технологий, языков программирования и инструментов, которые используются совместно для разработки программного обеспечения.

Токен – это цифровой идентификатор, использующийся для предоставления доступа к ресурсам или выполнению операций.

Фреймворк – это готовый набор инструментов, функций, методов, который облегчает разработчикам написание собственных программных компонентов.

Цифровизация – это применение цифровых технологий и разработок в различных сферах деятельности для повышения эффективности и автоматизации труда.

Эндпоинт – это конечная точка веб-сервиса, чаще всего URL, через который клиентская часть приложения взаимодействует с серверной для выполнения операций.

ВВЕДЕНИЕ

В современных условиях стремительной цифровизации и внедрения новых технологий в бизнес-практику, управление персоналом приобретает стратегически важное значение. Одним из ключевых направлений в этой области становится эффективный контроль и учет рабочего времени сотрудников. Он напрямую влияет на производительность труда, качество выполнения рабочих обязанностей, соблюдение дисциплины, а также способствует повышению прозрачности процессов и снижению издержек.

Использование устаревших подходов, таких как бумажные журналы или даже электронные таблицы, уже не отвечает вызовам времени. Они не только неэффективны, но и подвержены человеческому фактору, что ведёт к ошибкам, задержкам в обработке информации и недостаточному уровню прозрачности. Это особенно критично для крупных организаций с распределённой структурой и высокими требованиями к безопасности данных.

Автоматизированные системы контроля и учёта рабочего времени (СКРВ) становятся оптимальным решением для цифровой трансформации кадровых процессов. Такие системы позволяют организовать централизованный сбор информации, упрощают ведение отчётности, минимизируют трудозатраты на административные операции и предоставляют возможность выгрузки аналитических данных в различных форматах, включая готовые шаблоны Excel.

Особенно остро вопрос внедрения защищённых СКРВ стоит в организациях оборонного сектора России. В условиях требований к информационной безопасности и импортозамещению здесь важна не только функциональность, но и соответствие программного обеспечения ряду критериев: локализация хранения данных во внутреннем контуре, возможность работы на отечественных операционных системах (например, Astra Linux, РЕД ОС), а также гарантированное отсутствие каналов утечки информации при обмене данными между подразделениями.

Создание отечественной системы, учитывающей специфику конкретного предприятия, позволит не только автоматизировать сбор сведений о фактическом времени работы сотрудников, но и отслеживать отклонения, формировать отчеты для расчета заработной платы и принимать своевременные управленческие меры в отношении нарушителей трудовой дисциплины.

Целью данной выпускной квалификационной работы является проектирование и реализация автоматизированной системы учета и контроля рабочего времени персонала, адаптированной под нужды конкретной организации и соответствующей современным требованиям безопасности и эффективности.

Результатом станет программный продукт, способный формировать отчетность по каждому сотруднику в виде Excel-файлов, содержащих сведения о фактически отработанных часах, а также предоставить инструменты аналитики для руководства и бухгалтерии.

Для реализации поставленной цели были определены следующие задачи:

1. Провести исследование существующих программных решений в области СКРВ, выделить их функциональные особенности, достоинства и недостатки.
2. Сформировать перечень требований на основе анализа бизнес-процессов организации и определить цели автоматизации.
3. Подобрать оптимальные программно-аппаратные средства, соответствующие требованиям по безопасности, производительности и масштабируемости.
4. Разработать архитектурную модель системы с учетом распределённой структуры и перспектив расширения.
5. Сформировать алгоритмы сбора, обработки и хранения данных о рабочем времени.

6. Реализовать программное решение, обеспечивающее взаимодействие между пользователями и системой.
7. Провести испытания разработанной системы, оценить точность учёта и удобство использования.
8. Проанализировать экономическую эффективность внедрения системы и её влияние на оптимизацию труда.

Отличительной особенностью данной работы является реализация модели учета рабочего времени, ориентированной на специфические условия функционирования предприятия и интеграции с существующими системами документооборота.

Практическая значимость проекта заключается в снижении издержек на обработку информации, сокращении количества ошибок при учёте, повышении прозрачности работы персонала, а также в улучшении качества управленческих решений за счёт наличия достоверной и своевременной аналитики.

Результаты разработки были представлены на внутреннем семинаре предприятия и получили положительные отклики от специалистов отделов кадров, ИТ и бухгалтерии, что подтверждает актуальность и применимость предложенного решения в реальных условиях.

ГЛАВА 1. ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Общие принципы построения систем контроля рабочего времени

СКРВ представляют собой комплекс программно-технических средств, которые предназначены для автоматизации процесса фиксации и анализа времени, проведенного сотрудниками на рабочем месте, с целью повышения эффективности управления трудовыми ресурсами и соблюдения трудовой дисциплины.

Принцип действия автоматизированной СКРВ предполагает:

- Автоматизированное временное протоколирование;
- Пространственную фиксацию;
- Информационную оперативность;
- Формирование и представление статистики;
- Хранение и использование информации с помощью баз данных.

Различают несколько основных современных принципов функционирования систем контроля рабочего времени.

1.1.1. Видеонаблюдение

Суть данного метода контроля рабочего времени состоит в установке оборудования, которое фиксирует изображения, и, дополнительно, звук в помещении. Таким образом, можно отслеживать, где находится в данный момент времени сотрудник, исполняет ли он свои должностные обязанности, явился на работу вовремя, когда покинул рабочее место, как долго находился на обеденном перерыве и т.д. Но у этого метода есть существенные недостатки. Используя подобную систему, руководство не имеет возможности оценить продуктивность персонала, потому что затраты ресурсов на изучение и оценку всех видеозаписей по всем сотрудникам сильно превышают экономический эффект, полученный от подобного анализа. К тому же, сотрудник, зная о местонахождении камер видеонаблюдения, может сознательно находиться в «слепой зоне».

1.1.2. Биометрия

Метод контроля рабочего времени с применением биометрических идентификаторов основан на использовании уникальных физиологических характеристик человека. Принцип работы заключается в том, что сотрудник отмечает начало и завершение рабочего дня с помощью биометрической аутентификации. В роли идентификаторов могут выступать отпечатки пальцев, геометрия лица, рисунок вен ладони, скан радужной оболочки глаза и другие параметры, не поддающиеся подделке.

Как правило, в качестве аппаратной базы применяются специализированные биометрические терминалы или сканеры. Они фиксируют биометрическое изображение, оцифровывают его, а затем сравнивают с ранее зарегистрированными шаблонами в базе данных предприятия.

Среди основных преимуществ таких систем можно выделить:

- высокую точность идентификации, исключая возможность "передачи пропуска";
- повышенный уровень безопасности – контроль доступа невозможен без физического присутствия конкретного сотрудника;
- удобство использования – отсутствует необходимость в носимых пропусках, карточках или кодах;
- оперативность – идентификация осуществляется за доли секунды.

Однако у технологии есть и ряд ограничений:

- при утечке биометрических данных их невозможно "перегенерировать" как обычный пароль, что несёт потенциальные риски;
- такие системы не позволяют оценивать продуктивность сотрудника в течение дня – только факт присутствия;
- чувствительность к внешним условиям — загрязнение датчика или освещение могут повлиять на точность распознавания.

В целом, биометрические решения оправданы на объектах с повышенными требованиями к контролю доступа, однако требуют взвешенного подхода к вопросам информационной безопасности и оценки эффективности использования.

1.1.3. Электронные карты

Пропускной метод контроля рабочего времени основан на использовании персонализированных электронных карт или программируемых ключей, которые выдаются каждому сотруднику. С их помощью осуществляется вход на территорию предприятия, доступ в отдельные зоны или к конкретному оборудованию. Каждое использование пропуска фиксируется в системе контроля доступа через специальные терминалы, оснащённые считывателями или магнитными сенсорами.

Технология широко распространена благодаря своей универсальности и простоте внедрения. Систему легко масштабировать под любое количество сотрудников, а пропуска могут быть настроены индивидуально в зависимости от должностных обязанностей – вплоть до ограничения по времени или доступу к определённым объектам. Интеграция с корпоративными системами учёта позволяет получать статистику по перемещениям и пребыванию персонала на объекте.

Несмотря на свои плюсы, такой подход имеет ряд критических уязвимостей. Основная из них – высокий риск несанкционированного использования: карту можно потерять, украсть или сознательно передать другому человеку, что ставит под угрозу достоверность данных. Кроме того, система не учитывает нюансов рабочего процесса, таких как перемещения по служебной необходимости, обеденные перерывы или краткосрочные выезды по внутренним поручениям – всё это может исказить реальную картину активности сотрудника.

Таким образом, хотя пропускные системы и остаются одними из самых популярных благодаря низкому порогу внедрения и функциональной

гибкости, они требуют дополнительных механизмов контроля (например, видеонаблюдение или биометрия), чтобы повысить точность и надёжность учета рабочего времени.

1.1.4. Программные системы

В организациях, где каждому сотруднику выделено индивидуальное рабочее устройство, целесообразно внедрение программных решений для автоматизированного контроля трудовой активности. Такие системы позволяют отслеживать начало и завершение рабочего дня на основе событий включения и выключения компьютера, вести подробную статистику по каждому пользователю, а также анализировать занятость в режиме реального времени.

Программное обеспечение данного класса отличается высокой адаптивностью и широким спектром возможностей: от учёта времени активности до мониторинга посещаемых веб-ресурсов, использования прикладных программ и взаимодействия с корпоративными сервисами. Многие из таких решений поддерживают интеграцию с системами управления проектами, бухгалтерскими модулями и корпоративными мессенджерами, что повышает прозрачность бизнес-процессов и облегчает администрирование.

Однако существует и ряд ограничений. Подобные инструменты плохо подходят для учёта занятости мобильных сотрудников, выездных специалистов и работников, использующих в своей деятельности устройства, не поддерживающие установку соответствующего ПО (например, планшеты или личные смартфоны). Это делает их неуниверсальным решением в условиях распределённой или гибридной модели работы.

1.2. Анализ существующих решений

Для разработки автоматизированной системы учёта и контроля рабочего времени сотрудников необходимо провести анализ существующих решений.

Это позволит выявить лучшие практики, определить возможные недостатки и разработать более эффективное решение, учитывающее специфику проекта.

1.2.1. Yaware.TimeTracker

Это система для автоматического учёта рабочего времени сотрудников, которая отслеживает активность на компьютере, включая использование приложений и сайтов.

Преимущества:

- Автоматический учёт времени, что позволяет минимизировать человеческие ошибки и обеспечить точность данных.
- Возможность генерации отчётов, что упрощает анализ производительности сотрудников и планирование задач.
- Интеграция с популярными CRM-системами, что способствует более эффективному управлению бизнес-процессами.
- Наличие мобильного приложения для отслеживания времени работы сотрудников вне офиса.

Недостатки:

- Ограниченная функциональность для крупных предприятий, что может быть критично для организаций с большим количеством сотрудников и сложными бизнес-процессами.
- Отсутствие поддержки российских дистрибутивов, таких как Astra Linux, что может быть важным фактором в условиях повышенных требований к безопасности данных.

1.2.2. StaffCop

Это программа для мониторинга активности сотрудников, которая позволяет отслеживать использование компьютеров, интернета и приложений.

Преимущества:

- Широкий функционал для контроля сотрудников, что позволяет гибко настраивать параметры мониторинга и анализа активности.

- Возможность блокировки нежелательных ресурсов, что помогает повысить продуктивность работы и обеспечить безопасность данных.
- Поддержка интеграции с системами управления доступом и другими корпоративными приложениями.

Недостатки:

- Высокая стоимость, что может сделать решение недоступным для небольших компаний или проектов с ограниченным бюджетом.
- Сложность настройки и интеграции, что требует от IT-специалистов глубоких знаний и опыта работы с подобными системами.
- Высокие риски снижения мотивации сотрудников из-за чрезмерного контроля и мониторинга их деятельности.

1.2.3. Мегаплан

Это CRM-система с функцией учёта рабочего времени, которая позволяет планировать задачи и отслеживать их выполнение.

Преимущества:

- Интеграция с другими бизнес-процессами, что способствует более эффективному управлению проектами и задачами.
- Удобный интерфейс для управления задачами, что упрощает работу пользователей и повышает их удовлетворённость системой.

Недостатки:

- Ограниченные возможности для автоматического учёта времени, что может снизить точность и эффективность системы.
- Отсутствие поддержки локальных дистрибутивов, что может быть важно для компаний, работающих в условиях специфических требований к программному обеспечению.

1.2.4. Kickidler

Система для учета рабочего времени и контроля производительности сотрудников, включающая функции мониторинга экрана и анализа активности.

Преимущества:

- Возможность записи действий сотрудников в реальном времени.
- Гибкие настройки отчетности.
- Поддержка интеграции с другими корпоративными системами.

Недостатки:

- Высокая стоимость лицензий.
- Необходимость значительных ресурсов для хранения данных видеозаписей.
- Отсутствие поддержки отечественных дистрибутивов.

1.2.5. CrocoTime

Система автоматического учета рабочего времени с возможностью анализа продуктивности сотрудников.

Преимущества:

- Простота внедрения и использования.
- Генерация детальных отчетов.
- Поддержка мобильных устройств.

Недостатки:

- Ограниченные возможности для крупных предприятий.
- Отсутствие функций глубокого мониторинга активности.

Сводная информация по существующим решениям и критериям сравнения представлена в Таблице 1.

Таблица 1. Сводная информация по существующим решениям и критериям сравнения

Критерий	Yaware.TimeTracker	StaffCop	Мегаплан	Kickidler	CrocoTime
Автоматический учёт времени	Да	Да	Нет	Да	Да
Безопасность	Шифрование и журналирование	Сертификат ФСТЭК, контроль мессенджеров, имеет DLP	Нет	Шифрование и журналирование	Шифрование и журналирование
Совместимость с операционными системами	Windows	Windows, Astra Linux	Windows	Windows	Windows
Масштабируемость	Да	Да	Не предназначен для массового использования	Да	Да
Открытый исходный код	Нет	Нет	Нет	Нет	Нет
Формирование отчетов	Аналитика, графики, отчеты в Excel/PDF	Аналитика, графики, отчеты в Excel/PDF	Отчеты только по задачам	Аналитика, графики, отчеты в Excel/PDF	Аналитика, графики, отчеты в Excel/PDF

Пояснения к критериям сравнения:

- Автоматический учет времени – возможность автоматического отслеживания активности и простоев сотрудников.
- Безопасность – наличие сертификатов безопасности, реализация Data Loss Prevention.
- Совместимость с операционными системами – возможность официально установить и использовать полный функционал ПО на той или иной ОС.
- Масштабируемость – возможность установки ПО на неограниченное количество рабочих мест, без потери функциональных возможностей и удобства использования.
- Открытый исходный код – наличие исходного кода приложения в открытом доступе.
- Формирование отчетов – возможность гибкой настройки и формирования отчетных документов, включая статистику по выбранным сотрудникам за произвольный период времени.

1.3. Выводы

Анализ существующих решений в области учета рабочего времени выявил ряд их преимуществ и недостатков. Среди достоинств можно выделить автоматизированный сбор данных, формирование отчетной документации и возможность интеграции с корпоративными информационными системами, например, с модулями расчёта заработной платы или управления персоналом. Однако, из недостатков стоит упомянуть, что большинство представленных на рынке систем не адаптированы к условиям предприятий с повышенными требованиями к информационной безопасности. В частности, существенными недостатками являются отсутствие совместимости с отечественными операционными системами, такими как Astra Linux или РЕД ОС, что делает невозможным их использование в организациях, работающих в соответствии

с политикой импортозамещения. Кроме того, закрытый исходный код не позволяет проводить аудит безопасности, а ограниченная масштабируемость затрудняет внедрение подобных решений на предприятиях с распределенной инфраструктурой.

На основании всего вышесказанного можно сделать вывод о том, что разработка собственной системы учёта рабочего времени обусловлена необходимостью создания программного продукта, отвечающего современным требованиям к защищённости, гибкости и совместимости с российскими ИТ-платформами.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ РЕШЕНИЯ

2.1. Требования к разрабатываемому решению

В процессе создания эффективной системы учета и контроля рабочего времени сотрудников ключевым этапом является формирование требований к разрабатываемому решению. На этом этапе важно учесть специфику деятельности организации, актуальные стандарты информационной безопасности, а также современные тенденции в автоматизации управления персоналом. В следующей части будут подробно рассмотрены основные функциональные требования, определяющие архитектуру и ключевые особенности разрабатываемого программного продукта.

2.1.1. Функциональные требования

Функциональные требования к системе описаны в Таблице 2.

Таблица 2. Функциональные требования к системе

№	Название требования	Описание требования	Роль доступа
1	Система аутентификации и авторизации	Система включает в себя: • Матрица доступа пользователей (администратор, менеджер, сотрудник); • Аутентификация с использованием JWT-токенов • Управление сессиями пользователей; • Контроль доступа к функциям системы на основе ролей.	Администратор Менеджер Сотрудник

2	Модуль учета рабочего времени	Модуль включает в себя: • Автоматическое отслеживание активности пользователя; • Мониторинг активных приложений и окон; • Фиксация времени начала и окончания рабочего дня; • Определение периодов активности и простоя.	Администратор Менеджер Сотрудник
3	Система мониторинга активности	Система включает в себя: • Отслеживание использования приложений в реальном времени • Сбор данных о заголовках окон и процессах • Автоматическая категоризация активности (работа, перерывы, встречи) • Формирование временных линий активности сотрудников	Администратор Менеджер Сотрудник
4	Модуль отчетности и аналитики	Модуль включает в себя: • Генерация детализированных отчетов по периодам	Администратор Менеджер

		• Статистика по отделам и сотрудникам • Экспорт данных в формат Excel • Визуализация данных в виде диаграмм и графиков • Расчет показателей продуктивности	
5	Модуль администрирования системы	Включает в себя: • Управление пользователями и их ролями • Создание и редактирование профилей сотрудников • Настройка параметров отслеживания • Управление структурой отделов	Администратор
6	Управление учетной записью	Включает в себя: • Персональный кабинет пользователя • Настройки уведомлений и предпочтений	Администратор Менеджер Сотрудник

2.1.2. Требования к безопасности

Реализация системы учета рабочего времени требует многоуровневой архитектуры безопасности, обеспечивающей защиту данных и устойчивость к различным атакам и угрозам.

В первую очередь система должна обеспечивать надежную аутентификацию и авторизацию пользователей с применением криптографически стойких алгоритмов хеширования паролей, таких как bcrypt. Для управления сессиями необходимо реализовать токены с ограниченным временем жизни, а также многоуровневую систему контроля доступа на основе ролей, которая ограничивает права пользователей и защищает административные функции от несанкционированного использования.

Данные должны храниться в зашифрованном виде, а все сетевые соединения — осуществляться через защищенный протокол HTTPS с использованием современных стандартов шифрования. Входные данные должны проходить обязательную валидацию для предотвращения внедрения вредоносного кода, а взаимодействие с базой данных — осуществляться через объектно-реляционное отображение для защиты от SQL-инъекций [16].

2.1.3. Требования к надежности

Требования к надежности разрабатываемой системы учета и контроля рабочего времени предусматривают обеспечение высокой отказоустойчивости, производительности, масштабируемости и доступности, а также сохранность и целостность данных.

Архитектура системы должна быть построена на микросервисном принципе, что позволит изолировать компоненты и минимизировать влияние сбоев одного сервиса на работу всего решения. В случае недоступности отдельных сервисов система должна обеспечивать плавное деградирование функционала (graceful degradation) с сохранением основных возможностей.

Для повышения устойчивости необходимо реализовать автоматическое восстановление после сбоев и механизмы повторных попыток выполнения критически важных операций.

По требованиям к производительности время отклика API не должно превышать 200 миллисекунд для основных операций, а система должна

поддерживать одновременную работу до 100 пользователей без снижения качества обслуживания.

Для оптимизации работы с базой данных следует использовать индексы и кэширование часто запрашиваемой информации, что позволит снизить нагрузку и ускорить обработку запросов.

Необходимо обеспечить возможность масштабируемости системы и добавление новых сервисов без полной остановки работы приложения.

2.1.4. Требования к составу и параметрам технических средств

Технические требования приведены в Таблице 3.

Таблица 3. Технические требования к разрабатываемому решению

№	Требование
1	Программное обеспечение должно функционировать под управлением операционной системы Astra Linux версии 1.7 в исполнении «Смоленск» или «Воронеж».
2	Минимальные аппаратные требования: • Процессор: Intel Core i3-6100 / AMD A8-7600 или эквивалент (2 ядра, 2.0 GHz); • Оперативная память: 4 GB RAM; • Дисковое пространство: 2 GB свободного места; • Видеокарта: интегрированная графика Intel HD 520 или эквивалент; • Сеть: Ethernet 10 Mbps или Wi-Fi 802.11n; • Монитор с разрешением 1366x768 или выше. Рекомендуемые требования: • Процессор: Intel Core i5-8250U / AMD Ryzen 5 2500U или выше; • Оперативная память: 8 GB RAM; • Дисковое пространство: 5 GB свободного места;

	• Видеокарта: дискретная графика или интегрированная Intel UHD 620+; • Сеть: Gigabit Ethernet или Wi-Fi 802.11ac; • Монитор с разрешением 1920x1080 или выше;
3	Масштабирование по количеству пользователей. До 50 пользователей: • Сервер: 4 ядра, 8 GB RAM, 50 GB SSD; • Сеть: 100 Mbps. До 50-200 пользователей: • Сервер: 6 ядер, 16 GB RAM, 100 GB SSD; • Сеть: 500 Mbps; • Рекомендуется: Load Balancer. До 200-500 пользователей: • Сервер: 8+ ядер, 32 GB RAM, 200 GB SSD; • Отдельный сервер БД: 4+ ядра, 16 GB RAM, 100 GB SSD; • Сеть: 1 Gbps; • Обязательно: Load Balancer, кэширование.

2.2. Обоснование выбора подхода к разработке

Результат сравнения существующих систем показал, что для решения поставленных задач необходимо разработать гибкую, масштабируемую и современную архитектуру, а также подготовить функциональные требования. Для реализации автоматизированной системы учёта и контроля рабочего времени сотрудников можно рассмотреть два основных подхода: микросервисный и монолитный.

2.2.1. Микросервисный подход

Микросервисный подход в проектировании программных систем основан на декомпозиции приложения на набор мелких, автономных

компонентов (сервисов), каждый из которых решает строго ограниченный круг задач и взаимодействует с остальными через стандартизированные интерфейсы, как правило, по протоколу HTTP или через брокеры сообщений.

Среди основных преимуществ микросервисной архитектуры можно выделить:

- Горизонтальное масштабирование отдельных компонентов, что позволяет равномерно распределять вычислительную нагрузку и адаптироваться к изменению объёмов трафика.
- Изолированное обновление модулей, при котором изменения в одном сервисе не требуют остановки или пересборки всей системы.
- Упрощённое внесение изменений в архитектуру и бизнес-логику за счёт слабой связанности компонентов.

Тем не менее, вместе с преимуществами микросервисы приносят и ряд новых вызовов. Их использование предполагает более высокую сложность проектирования, администрирования и эксплуатации, особенно на этапах межсервисного взаимодействия и обеспечения целостности данных. Распределённая природа архитектуры требует решения задач координации, трассировки запросов, обработки ошибок в цепочках вызовов и организации защищённых каналов связи между сервисами.

К типичным недостаткам микросервисной архитектуры относятся:

- Увеличенные затраты на инфраструктуру, включая необходимость внедрения дополнительных инструментов для логирования, мониторинга и оркестрации сервисов (например, Prometheus, Grafana, Kubernetes).
- Сложность поддержки распределённых транзакций, особенно в случае, если сервисы используют разные базы данных или хранилища.

Наиболее эффективно микросервисная модель проявляет себя в информационных системах корпоративного масштаба, где требуется высокая степень масштабируемости, модульности и гибкости.

Таким образом, несмотря на возросшую инженерную сложность, микросервисная архитектура остаётся одним из самых перспективных решений для построения надёжных и гибких программных систем.

2.2.2. Монолитный подход

Монолитная архитектура представляет собой структуру программного обеспечения, при которой все функциональные модули приложения, от пользовательского интерфейса до логики работы с базой данных, объединены в одну общую кодовую базу и компилируются в единый исполняемый файл. Такой подход облегчает начальную разработку, позволяет быстрее развернуть продукт и минимизирует затраты на организацию межмодульного взаимодействия, поскольку компоненты взаимодействуют напрямую, без необходимости настройки сетевых протоколов или API.

Основными преимуществами монолитных приложений являются:

- Простота проектирования и внедрения. Монолит легче реализовать в условиях ограниченных ресурсов, что делает его привлекательным выбором для стартапов и небольших команд разработки.
- Минимальные накладные расходы. Отсутствие необходимости в сложной настройке взаимодействия между сервисами снижает технические и временные затраты.
- Упрощённое тестирование. Общая кодовая база позволяет быстрее проводить интеграционное и модульное тестирование без необходимости эмуляции внешних сервисов.

Однако, с ростом объема функционала и увеличением числа пользователей возникают ограничения, делающие монолитную архитектуру менее эффективной:

Среди ключевых недостатков можно отметить:

- Масштабировать отдельные модули невозможно, так как необходимо увеличивать ресурсы для всего приложения, что не всегда оправдано.
- Изменения, внесённые в одну часть системы, могут вызвать непредвиденные сбои в других, что усложняет тестирование и поставку новых версий.
- В рамках одного монолита чаще всего применяется единый стек, что сдерживает использование более подходящих или современных решений для отдельных задач.
- Даже незначительное обновление требует повторной сборки и развертывания всего приложения, что увеличивает риск ошибок и время отклика на изменения.

В результате, монолитная архитектура оптимальна для небольших проектов и организаций с ограниченными требованиями к масштабируемости и отказоустойчивости. Однако при росте объема системы или расширении команды разработчиков такой подход становится менее гибким, уступая по ряду параметров микросервисной архитектуре.

2.3. Обоснование архитектурных решений

На основании представленных преимуществ и недостатков был выбран микросервисный подход для реализации приложения по учету и контролю рабочего времени сотрудников.

Для разработки системы выбран стек технологий, с которым будет возможно добиться ожидаемых результатов, а приложение будет отвечать современным требованиям к надежности, безопасности и эффективности.

2.3.1. Клиентская часть

Клиентская часть веб-приложения представляет собой программный компонент, отвечающий за взаимодействие с пользователем и визуальное отображение информации. Она реализует пользовательский интерфейс, обеспечивая удобный и интуитивно понятный доступ к функционалу приложения. Основной задачей клиентской части является формирование запросов к серверу и обработка полученных ответов, что позволяет динамически обновлять содержимое без необходимости полной перезагрузки страницы.

Для реализации интерфейса используется Vue.js – прогрессивный JavaScript-фреймворк, оптимально подходящий для создания одностраничных приложений. Навигация между разделами реализована с помощью Vue Router, что позволяет быстро переключаться между представлениями без обновления страницы.

Vueх используется как централизованное хранилище состояния приложения. Он упрощает управление данными и гарантирует их согласованность между компонентами.

Для обмена данными с сервером применяется Axios – легковесная библиотека для выполнения HTTP-запросов, обеспечивающая удобную обработку ответов и ошибок.

Для локального хранения данных на клиенте используется LocalStorage, позволяющий сохранить сессионную информацию и поддерживать доступ к данным в офлайн-режиме. Периодическая синхронизация с сервером поддерживает актуальность данных и сокращает время отклика интерфейса.

2.3.2. Серверная часть

Серверная часть веб-приложения представляет собой ключевой компонент, отвечающий за обработку запросов от клиентов, выполнение бизнес-логики и управление хранением данных. В отличие от клиентской стороны, которая обеспечивает взаимодействие с пользователем и визуализацию информации, серверная часть функционирует на стороне

сервера и остается невидимой для конечного пользователя, обеспечивая при этом корректное и безопасное выполнение всех операций, необходимых для работы приложения.

Основу серверной архитектуры составляет FastAPI – асинхронный Python-фреймворк, известный своей высокой производительностью и удобством для разработки RESTful API. В качестве сервера используется Uvicorn, совместимый с ASGI, что позволяет обрабатывать большое количество одновременных подключений с минимальными задержками.

Для хранения и управления данными выбрана PostgreSQL – мощная реляционная СУБД, способная эффективно справляться с большими объёмами информации и сложными запросами. Вся информация о пользователях, а также статистика их активности сохраняется в этой системе.

Работа с базой данных реализована через SQLAlchemy, предоставляющую механизм ORM, благодаря которому можно взаимодействовать с таблицами как с Python-объектами, избавляясь от необходимости вручную писать SQL-запросы.

Аутентификация и авторизация реализованы с использованием JWT (JSON Web Tokens). Это решение позволяет безопасно управлять пользовательскими сессиями, защищая доступ к данным и API-методам.

2.3.3. Сетевые технологии

Для обеспечения эффективного взаимодействия между клиентской и серверной частями веб-приложения, а также для гарантирования безопасности и высокой производительности, в разрабатываемом решении применяются современные сетевые технологии.

RESTful API используется для обеспечения взаимодействия между клиентской и серверной частями системы. RESTful API позволяет легко обмениваться данными и обеспечивает гибкость при разработке и масштабировании системы.

HTTPS для защиты передаваемых данных между клиентом и сервером используется протокол HTTPS, который обеспечивает шифрование и аутентификацию, предотвращая несанкционированный доступ к конфиденциальной информации.

WebSockets для реализации двусторонней коммуникации в реальном времени между клиентом и сервером можно использовать WebSockets. Это особенно полезно для уведомлений и обновлений данных в режиме реального времени.

CORS (Cross-Origin Resource Sharing) это механизм браузера, который позволяет определить список ресурсов, к которым страница может получить доступ.

2.3.4. Дополнительные инструменты

Docker – это открытая платформа контейнеризации с открытым исходным кодом, используемая для упаковки, доставки и запуска приложений в контейнерах. Данный инструмент для автоматизации развертывания приложений позволяет быстро и легко развертывать приложения в различных средах.

Docker Compose – это инструмент для определения и запуска многоконтейнерных Docker-приложений [1]. Он упрощает разработку, развёртывание и тестирование сложных проектов, где используются несколько сервисов. С помощью файла `docker-compose.yml` можно описать конфигурацию всех сервисов, сетей и томов, необходимых для приложения, и запустить их одной командой.

ГЛАВА 3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ

3.1. Описание функциональности

В соответствии с требованиями, предъявляемыми к системе, было разработано веб-приложение, позволяющее осуществлять учет и контроль рабочего времени сотрудников путем мониторинга активности каждого пользователя системы и возможности создания отчетных документов за заданный период времени.

Структура проекта представлена на Рис. 1.

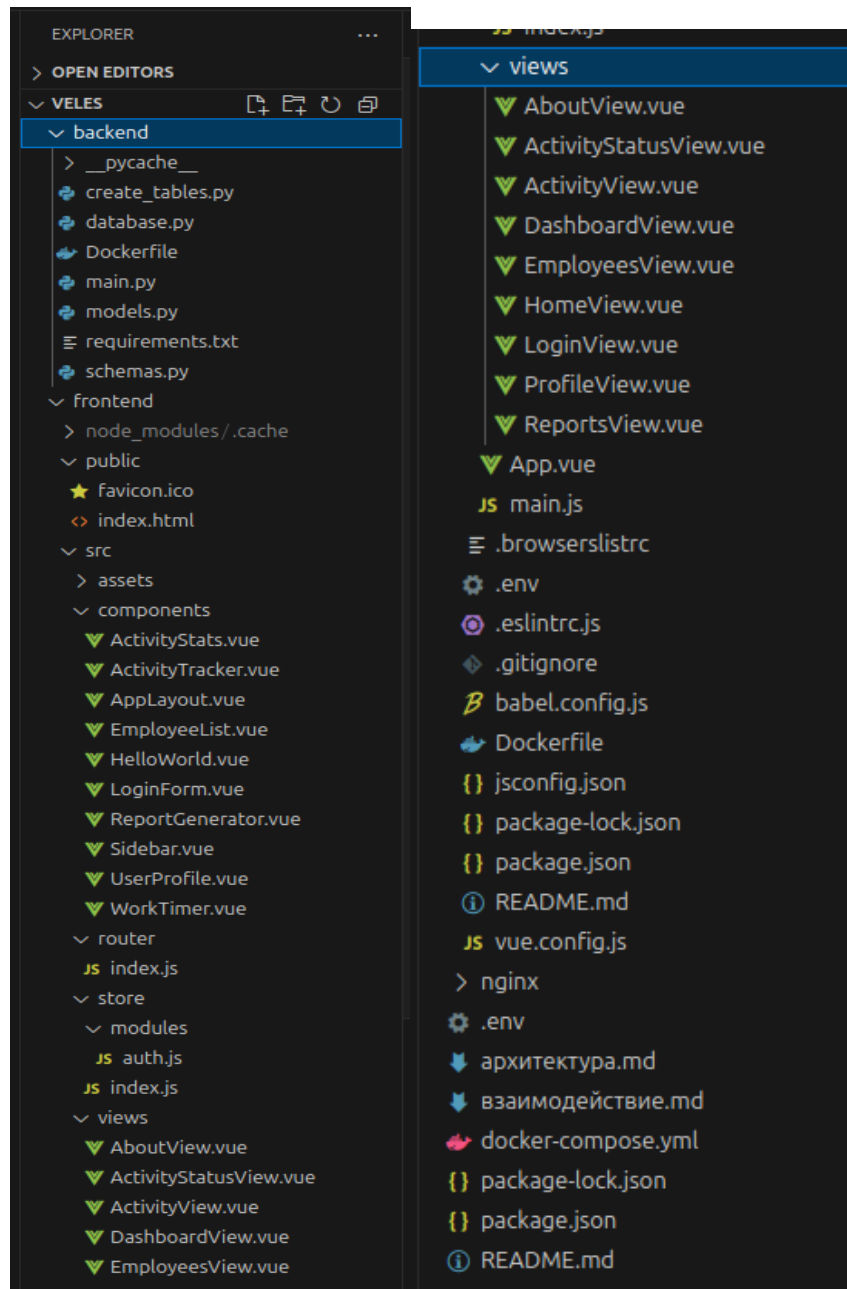


Рис. 1. Структура проекта

3.1.1. Система аутентификации и авторизации

Для безопасного хранения пользовательских паролей применяется алгоритм `bcrypt`, реализованный с помощью библиотеки `passlib`. Процесс аутентификации построен на механизме JWT, где генерация и валидация токенов осуществляется с помощью библиотеки `python-jose`. JWT-токены содержат зашифрованную информацию о пользователе и имеют ограниченный срок действия, что минимизирует риски несанкционированного доступа.

На стороне сервера реализован специальный эндпоинт `/token`, который принимает учетные данные пользователя (`email` и пароль), а в случае успешной проверки возвращает JWT-токен для последующего использования в запросах. Для автоматической проверки подлинности каждого запроса в системе используется `middleware`-функция `get_current_user()`, которая извлекает и валидирует токен из заголовка авторизации, обеспечивая тем самым централизованный контроль доступа.

Клиентская часть реализована с использованием `Vue.js` и организует процесс аутентификации следующим образом: пользователь вводит свои данные в компоненте `LoginForm.vue`, после чего информация передается во `Vuex store` – централизованное хранилище состояния приложения. `Store` сохраняет полученные данные пользователя и токен в `localStorage` браузера, что позволяет сохранять сессию между перезагрузками страниц. Для предотвращения несанкционированного доступа к защищённым маршрутам используется `router guard`: при каждом переходе между страницами проверяется наличие и валидность токена. В зависимости от роли пользователя, компонент `AppLayout.vue` динамически отображает соответствующий интерфейс и доступные функции, реализуя таким образом разграничение прав внутри системы.

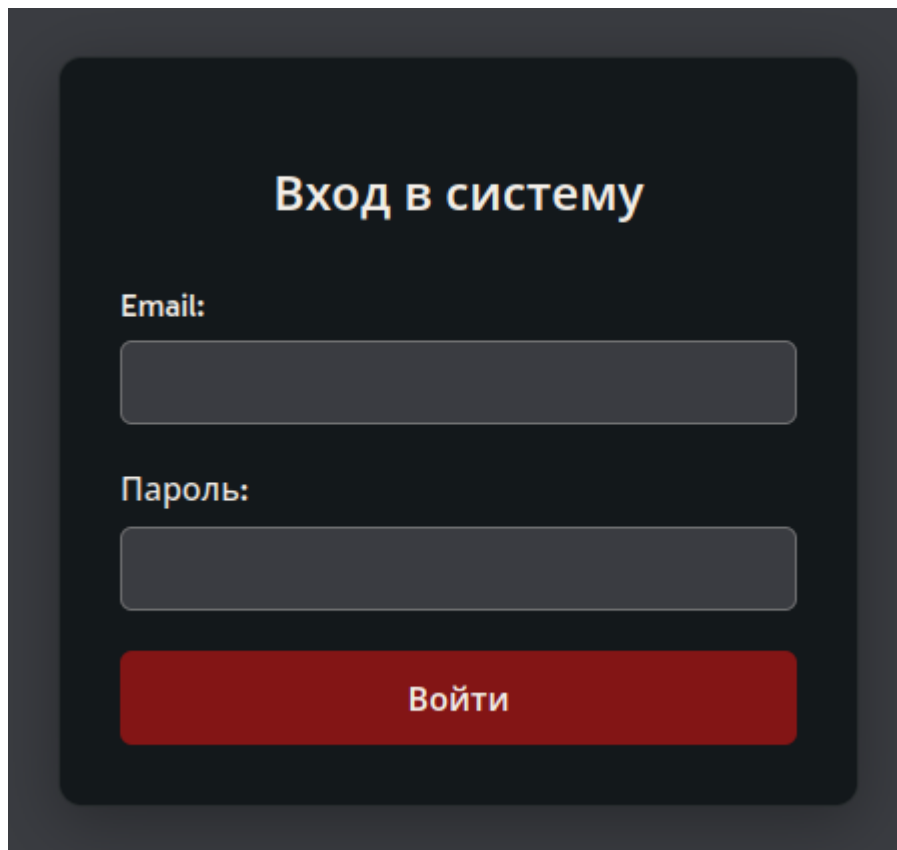


Рис. 2. Окно авторизации

3.1.2. Модуль учета рабочего времени

Модуль учета рабочего времени работает следующим образом: сотрудник нажимает кнопку «Начать рабочий день» в компоненте WorkTimer.vue, после чего запускается JavaScript-таймер, отслеживающий время активности. По завершении смены пользователь нажимает «Закончить рабочий день», и на сервер отправляется POST-запрос с временными метками.

Сервер принимает данные, рассчитывает продолжительность рабочего дня и сохраняет результат в базу данных. Итоговое время отображается в интерфейсе, обеспечивая сотруднику удобный и прозрачный контроль.

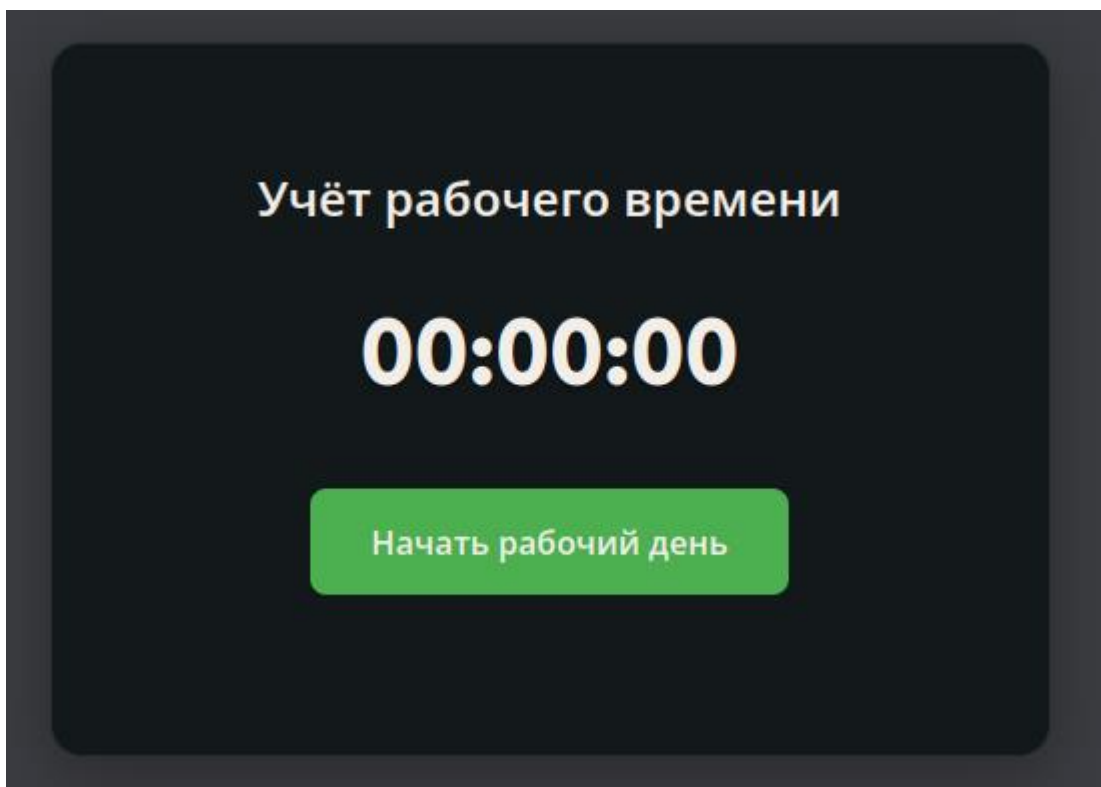


Рис. 3. Инструмент фиксации рабочего времени

3.1.3. Система мониторинга активности

Система мониторинга активности сотрудников реализует отслеживание использования приложений в режиме реального времени, сбор данных о заголовках окон и запущенных процессах, а также автоматическую категоризацию активности на рабочие периоды, перерывы и встречи. На основе собранной информации формируются временные линии активности, что позволяет получить детальную картину распределения рабочего времени и поведения сотрудников.

Работа системы начинается с автоматического запуска компонента ActivityTracker.vue при загрузке клиентского приложения. Каждые пять секунд фронтенд отправляет запрос к эндпоинту /current-activity, инициируя обновление данных о текущей активности пользователя.

Серверная часть, используя системные утилиты операционной системы Linux, определяет активное окно и извлекает информацию о приложении и

заголовке окна. Эти данные сохраняются в базе данных в виде записей ActivityLog, обеспечивая хранение и последующий анализ активности.

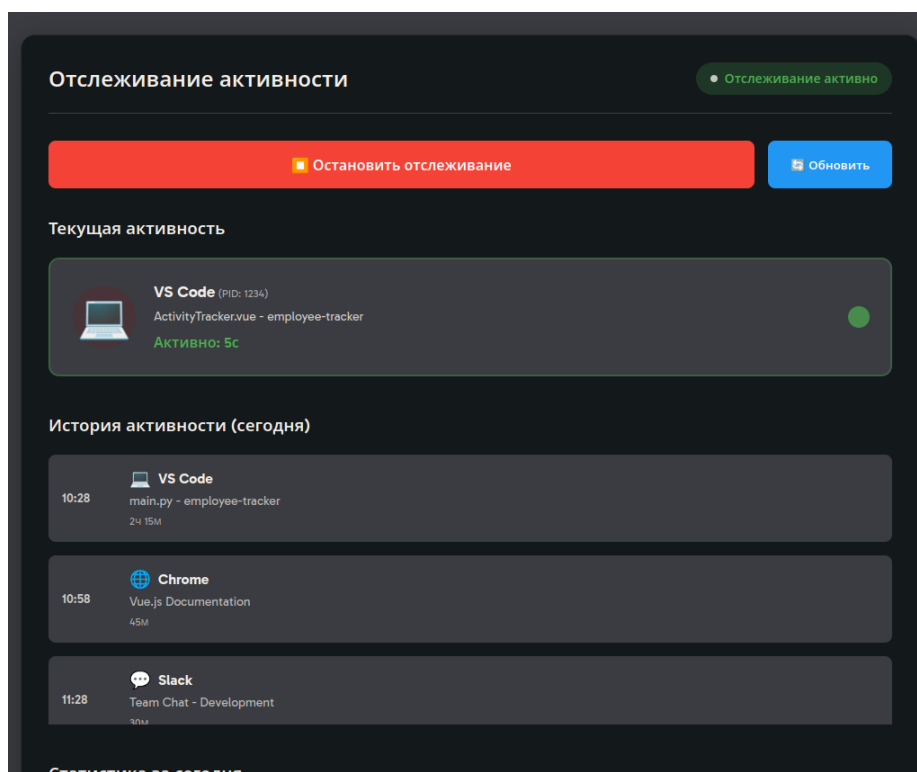


Рис. 4. Окно отслеживания активности

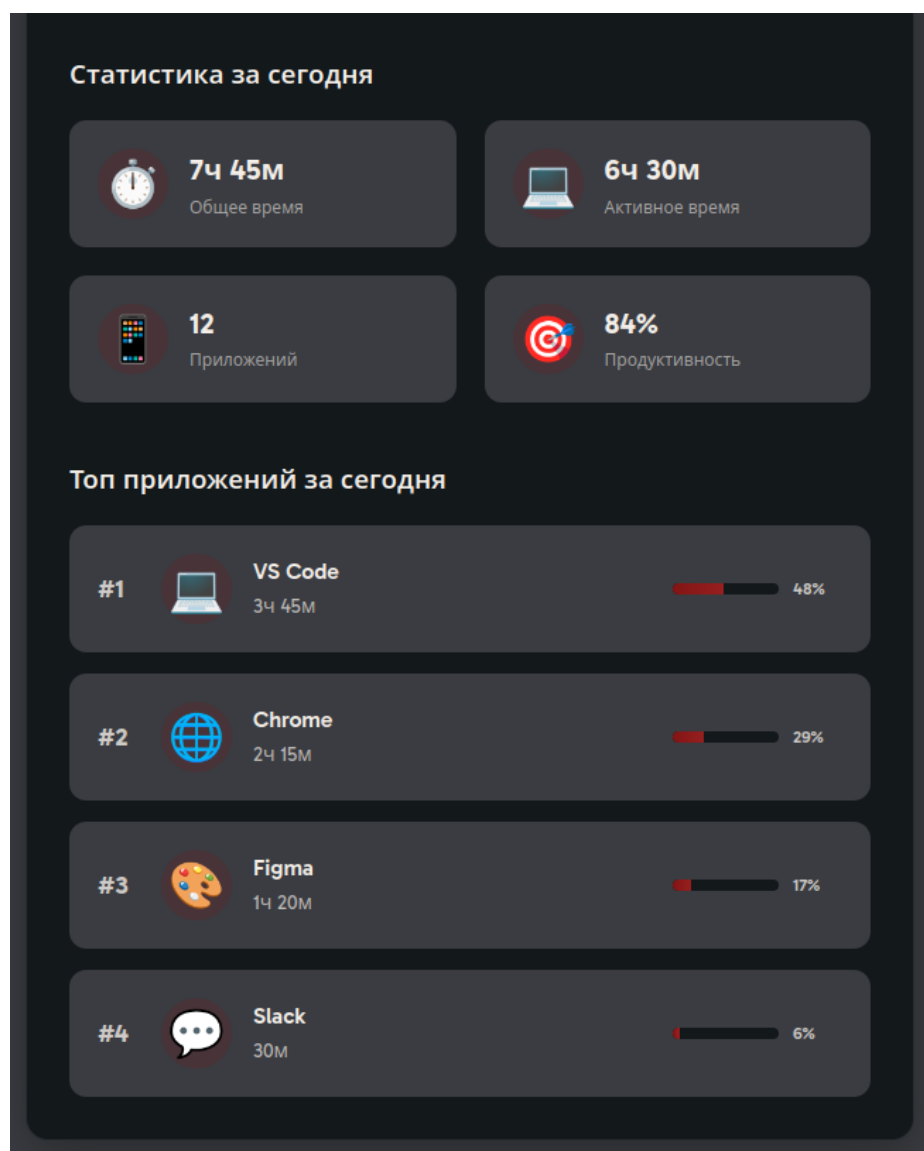


Рис. 5. Окно отслеживания статистики

3.1.4. Модуль отчетности и аналитики

Модуль генерации отчетов обеспечивает создание аналитических сводок за выбранные периоды, включая статистику по отделам и отдельным сотрудникам. В функционал модуля входит возможность экспорта данных в формат Excel и расчет показателей продуктивности, позволяя оценить эффективность работы персонала и выявить тенденции в распределении рабочего времени.

Работа модуля начинается с выбора пользователем периода и необходимых фильтров в интерфейсе компонента ReportGenerator.vue. После

этого формируется и отправляется запрос на серверный эндпоинт /report с параметрами фильтрации.

Серверная часть агрегирует данные из таблиц ActivityLog и Employee, группируя информацию по сотрудникам и отделам, и возвращает результат в формате JSON. Для экспорта в Excel используется библиотека openpyxl, которая формирует файл с отчетом, передаваемый клиенту в виде бинарного объекта (blob).

На стороне фронтенда автоматически инициируется скачивание созданного файла через сгенерированную ссылку, обеспечивая удобство и оперативность получения отчетной информации.

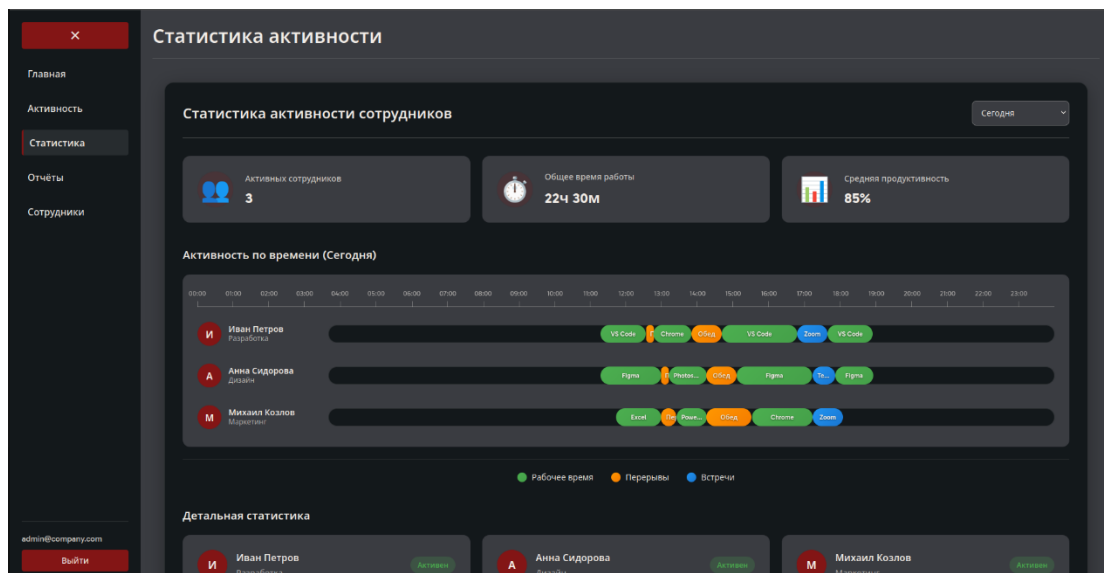


Рис. 6. Окно просмотра статистики по всем сотрудникам

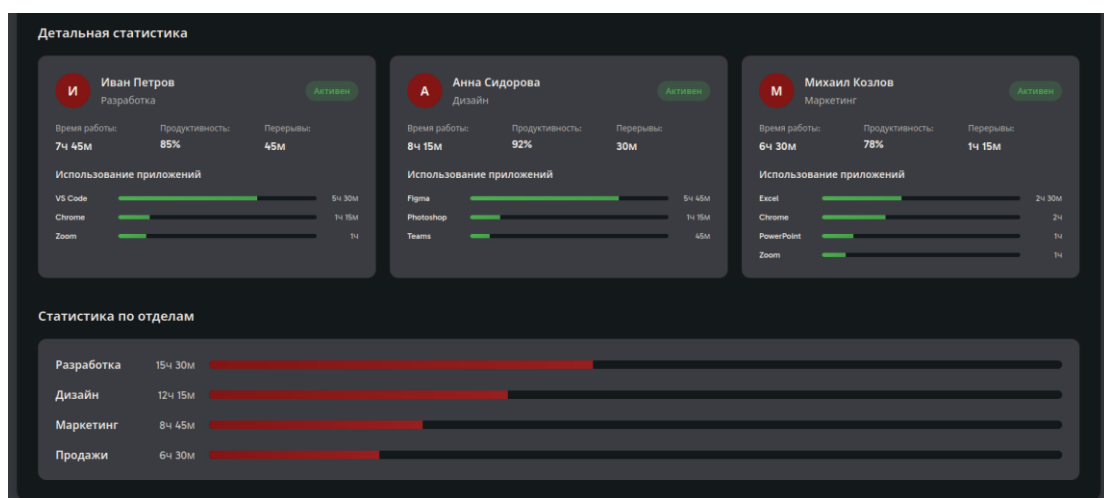


Рис. 7. Окно просмотра детальной статистики

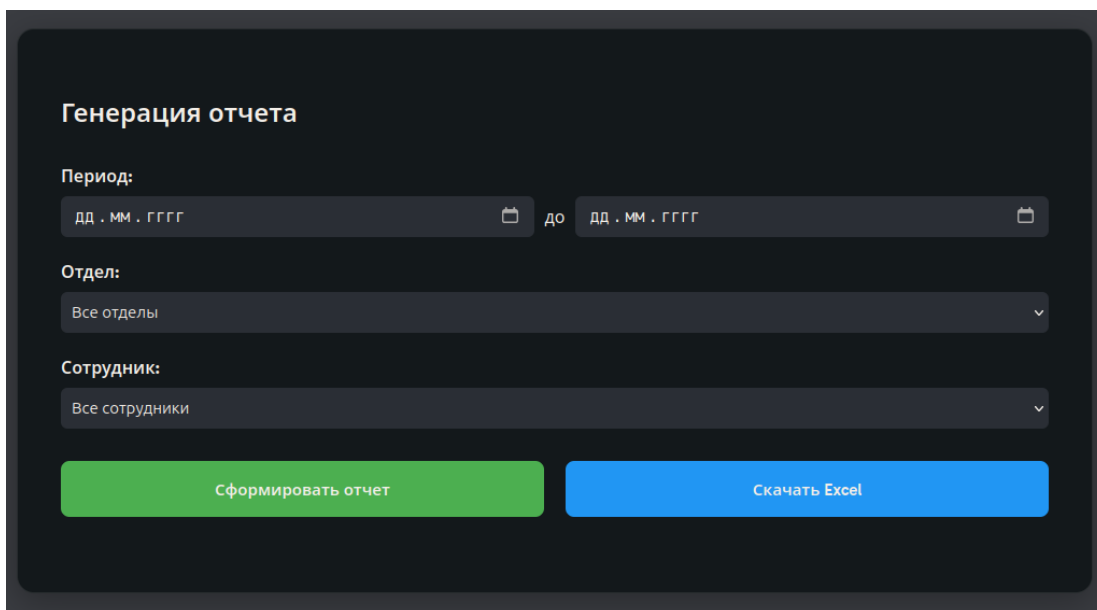


Рис. 8. Инструмент генерации отчетного документа

3.1.5. Модуль администрирования системы

Модуль обеспечивает комплексное администрирование учетных записей сотрудников, их ролей и параметров отслеживания рабочего времени, а также управление структурой отделов компании. В рамках данного модуля реализованы функции создания и редактирования профилей сотрудников. Настройка параметров отслеживания предоставляет возможность адаптировать систему под специфические требования различных подразделений и типов деятельности.

Работа модуля начинается с того, что администратор открывает страницу «Сотрудники» (`EmployeesView.vue`), доступ к которой контролируется с помощью `router guard`, проверяющего роль пользователя и разрешая доступ только администраторам и менеджерам.

После успешной проверки загружается список сотрудников посредством запроса к эндпоинту `/employees`. При необходимости создания нового пользователя открывается модальное окно, в котором вводятся данные сотрудника. После подтверждения данные отправляются на сервер через эндпоинт `/users`, где происходит проверка прав доступа и валидация информации. В случае успешной обработки создаются соответствующие

записи в таблицах users и employees базы данных, обеспечивая интеграцию учетных данных и профилей сотрудников.

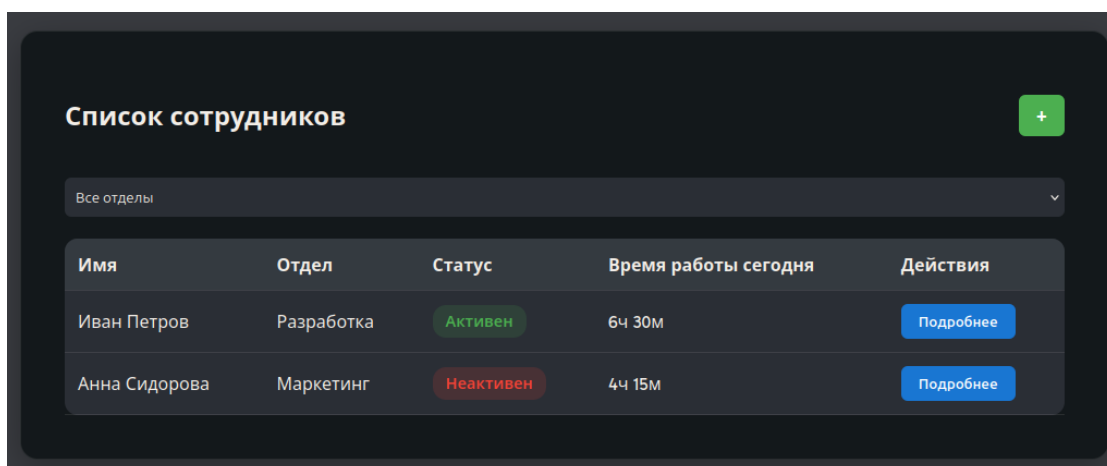


Рис. 9. Окно управления учетными записями

Создание пользователя

ФИО

Email (логин)

Пароль

Роль

Сотрудник

Отдел

Создать

Отменить

Рис. 10. Модальное окно создания пользователя

3.1.6. Управление учетной записью

Управление учетной записью организовано таким образом, чтобы обеспечить пользователю удобный доступ к персональной информации и настройкам системы.

При открытии страницы ProfileView.vue данные загружаются из Vuex store, который синхронизирован с localStorage, что позволяет сохранять состояние между сессиями и обеспечивать быстрый доступ к информации. В интерфейсе отображаются основные сведения о пользователе, включая ФИО, электронную почту, роль, отдел и статистику активности.

При необходимости редактирования профиля активируется соответствующий режим, позволяющий пользователю вносить изменения в свои данные.

Настройки уведомлений и параметры отслеживания также сохраняются в localStorage, что обеспечивает индивидуальную конфигурацию пользовательского опыта без задержек, связанных с сетевыми запросами.

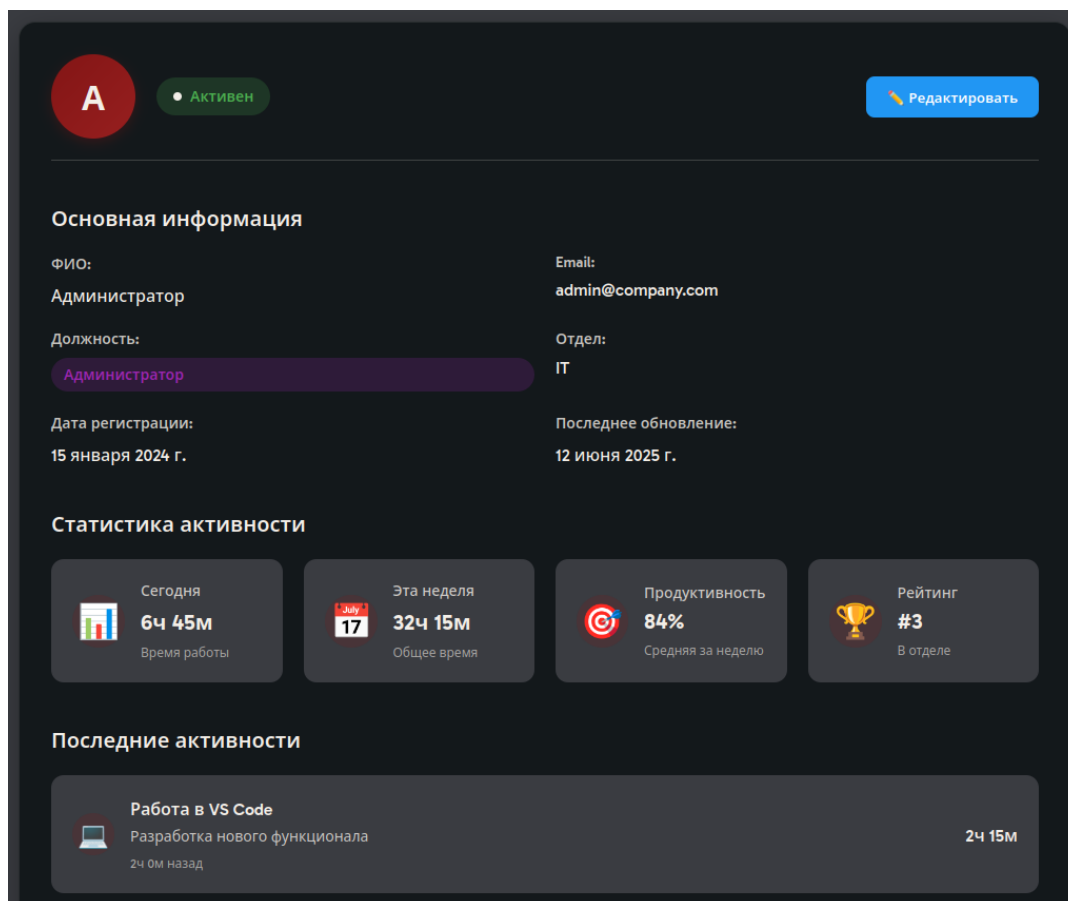


Рис. 11. Просмотр основной информации пользователя

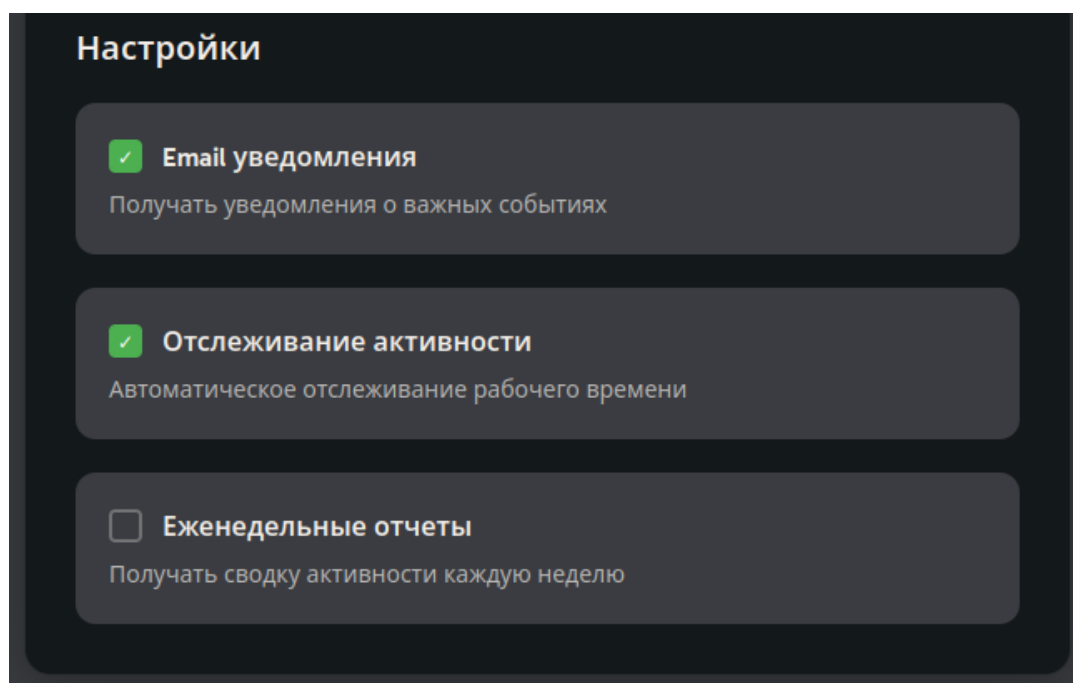


Рис. 12. Индивидуальные настройки пользователя

3.2. Обеспечение требований к безопасности

Основой системы стала аутентификация и авторизация, построенная на криптографически стойком алгоритме хеширования bcrypt для безопасного хранения паролей. Механизм JWT-токенов с ограниченным временем жизни (30 минут) минимизирует риски перехвата сессий, а ролевая модель доступа (RBAC) дифференцирует права пользователей, блокируя несанкционированные попытки доступа к функциям администрирования системы.

Защита данных реализована через шифрование информации в базе данных с использованием AES-256, что исключает компрометацию при утечке. Все сетевые взаимодействия защищены протоколом HTTPS с TLS 1.3, обеспечивающим целостность передаваемых данных. Для предотвращения инъекционных атак применен ORM SQLAlchemy, автоматически экранирующий пользовательский ввод, дополненный строгой валидацией входных параметров.

3.3. Обеспечение требований к надежности

В разработанной системе отказоустойчивость обеспечивается через многоуровневую архитектуру с четким разделением ответственности между компонентами. Микросервисная архитектура реализована посредством контейнеризации с использованием Docker Compose, где каждый сервис (frontend, backend, database) функционирует в изолированной среде [13]. Это обеспечивает независимость компонентов и предотвращает каскадные отказы.

Механизм graceful degradation реализован на уровне frontend-приложения через систему fallback-данных. В компоненте ActivityTracker.vue при недоступности API сервера система автоматически переключается на демонстрационные данные, позволяя пользователю продолжать работу с ограниченной функциональностью. Аналогичный подход применен в модуле статистики ActivityStats.vue, где при отсутствии соединения с сервером отображаются локально сохраненные данные.

Высокая доступность системы достигается через мониторинг состояния компонентов и автоматическое восстановление сервисов. В FastAPI реализованы health-check эндпоинты, которые позволяют внешним системам мониторинга отслеживать состояние приложения в реальном времени.

Оптимизация производительности достигается через использование асинхронного фреймворка FastAPI на backend, который обеспечивает высокую пропускную способность благодаря неблокирующим операциям ввода-вывода. Время отклика API оптимизировано через использование индексов в PostgreSQL на часто запрашиваемых полях (email, employee_id, start_time).

Горизонтальное масштабирование обеспечивается контейнерной архитектурой Docker, позволяющей легко увеличивать количество экземпляров сервисов в зависимости от нагрузки. Конфигурация docker-compose.yml поддерживает масштабирование через команды docker-compose scale, что позволяет динамически изменять количество работающих контейнеров.

База данных PostgreSQL настроена для поддержки кластерной конфигурации через репликацию master-slave, что обеспечивает распределение нагрузки чтения между несколькими узлами. Структура таблиц оптимизирована для горизонтального шардинга по employee_id, что позволяет распределять данные между несколькими серверами баз данных.

Транзакционная обработка критических операций обеспечивается через использование SQLAlchemy ORM с поддержкой ACID-транзакций PostgreSQL. В модуле создания пользователей реализована атомарная операция, которая создает записи в таблицах users и employees в рамках одной транзакции, предотвращая возникновение несогласованного состояния данных.

Механизм резервного копирования реализован через стандартные инструменты PostgreSQL (pg_dump) с автоматическим планированием через

cron. Скрипт `create_tables.py` включает проверки целостности данных при инициализации системы, предотвращая запуск с поврежденной базой данных.

Валидация данных осуществляется на нескольких уровнях: клиентская валидация в `Vue.js` компонентах, серверная валидация через `Pydantic` схемы в `FastAPI`, и ограничения целостности на уровне базы данных через `foreign key constraints`.

ГЛАВА 4. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ И ОЦЕНКА ЭФФЕКТИВНОСТИ

4.1. Тестирование приложения

Для тестирования разработанного приложения были проведены модульные, интеграционные и сквозные тесты. Результаты отражены в Таблице 4.

Таблица 4. Результаты тестирования

Тип тестов		Количество		Покрытие	Статус
Модульные	Клиентская часть	19	47	85%	Пройдены
	Серверная часть	28			
Интеграционные	Работа с API	12	23	78%	Пройдены
	Работа с БД	6			
	Клиент-серверная интеграция	5			
Сквозные		8		67%	Пройдены

Итого было проведено 78 тестов. Все критические функции протестированы и работают корректно. Система готова к развертыванию на тестовом стенде с рекомендацией по дальнейшему увеличению покрытия тестами.

4.2. Результаты проверок на тестовом стенде

После завершения этапа разработки и тестирования необходимо проверить систему на тестовом стенде. Для этой задачи использовался

компьютер, соответствующий минимальным аппаратным и программным требованиям приложения.

4.2.1. Разворачивание приложения

Перед запуском системы необходимо скачать архив с исходным кодом, а также проверить наличие Docker на стенде. В перспективе для «коробочной» версии приложения можно написать скрипт, который автоматически будет запускать скачивания зависимостей и распаковку архива с исходным кодом.

После выполнения описанных выше шагов необходимо открыть терминал, перейти в директорию проекта с помощью команды:

- `cd <путь к директории с проектом>.`

Запуск сборки docker-образов осуществляется командой:

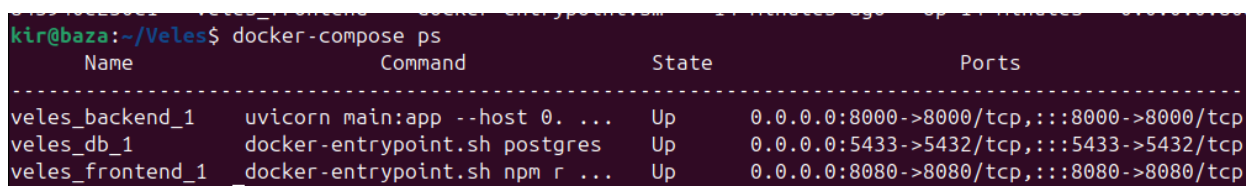
- `docker-compose up --build.`

В консоли появился вывод об успешном запуске контейнеров, времени сборки и адресе на сервере.

Для просмотра списка запущенных контейнеров можно использовать команду:

- `docker-compose ps.`

Если в процессе установки не возникло ошибок, как в моем случае, то вывод данной команды будет аналогичным Рис. 13.



Name	Command	State	Ports
veles_backend_1	uvicorn main:app --host 0. ...	Up	0.0.0.0:8000->8000/tcp, :::8000->8000/tcp
veles_db_1	docker-entrypoint.sh postgres	Up	0.0.0.0:5433->5432/tcp, :::5433->5432/tcp
veles_frontend_1	docker-entrypoint.sh npm r ...	Up	0.0.0.0:8080->8080/tcp, :::8080->8080/tcp

Рис. 13. Просмотр запущенных контейнеров

4.2.2. Работа с учетными записями

По умолчанию в базе данных создана учетная запись администратора, для первоначальной настройки системы. Для тестирования авторизации и

разграничения прав доступа согласно ролевой модели были созданы 2 новых учетных записи:

- Менеджер: manager@company.com
- Сотрудник: employee@company.com.

Процесс авторизации под каждой учетной записью оказался успешным. Для каждого пользователя с ролью manager и employee в боковой панели отображаются только доступные ему модули системы. Результат продемонстрирован на Рис. 14 и 15 соответственно.

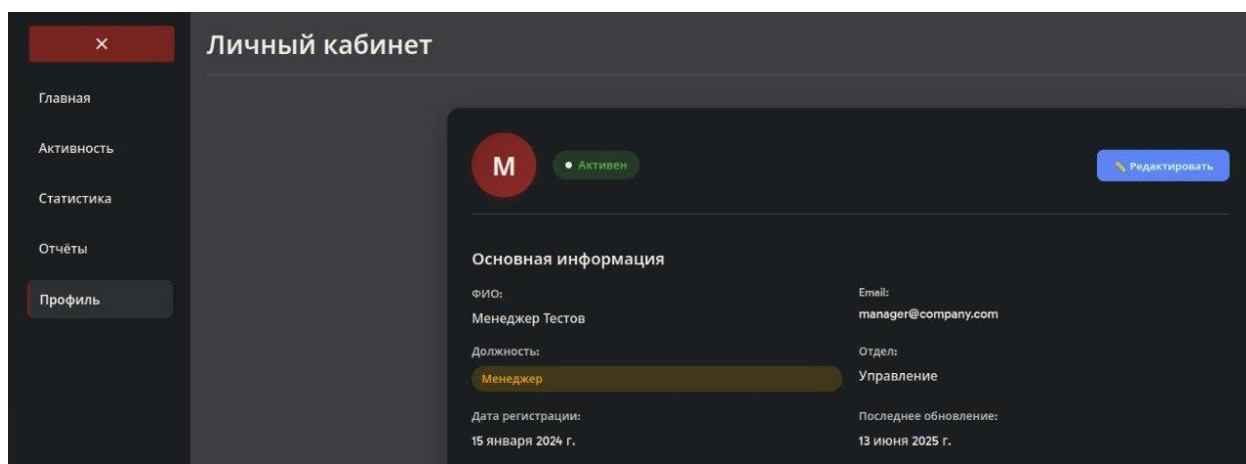


Рис. 14. Личный кабинет менеджера

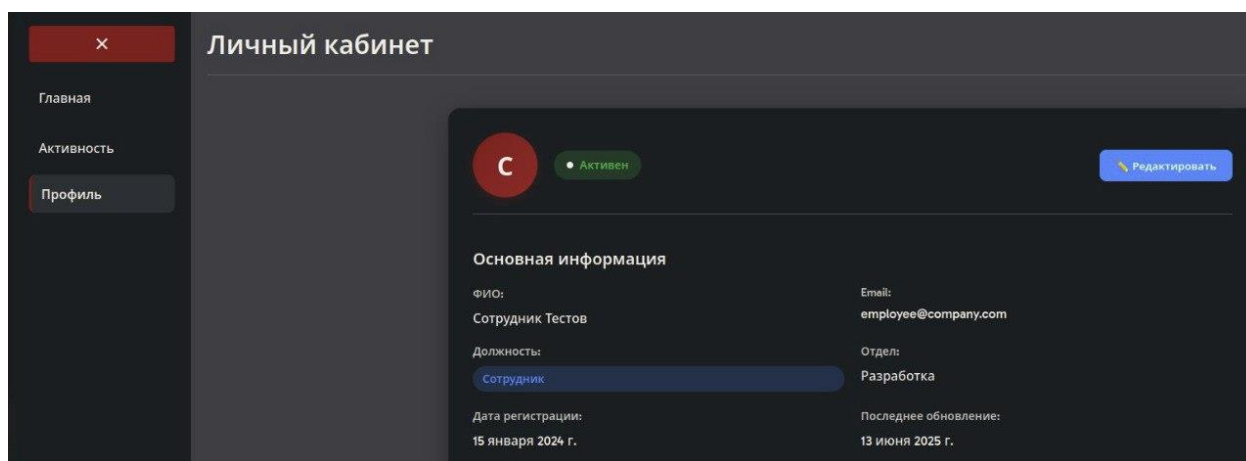


Рис. 15. Личный кабинет сотрудника

4.2.3. Генерация отчетов

Для тестирования работы механизма генерации отчетных документов под каждым созданным пользователем была совершена имитация рабочей деятельности. Результаты на Рис. 16.

The screenshot shows an Excel spreadsheet with the following data:

	A	B	C	D	E	F	G	H	I	J
1	Фамилия	Имя	Отчество	Отдел	Общее время работы	Активное время	Приложения			
2	Сотрудник	Тестов		Разработка	0ч 33м	0ч 17м	Firefox: 0ч 17м			
3	Менеджер	Тестов		Управление	1ч 12м	0ч 8м	Firefox: 0ч 8м			
4	Администратор			IT	4ч 45м	3ч 11м	Firefox: 0ч 17м VS Code: 0ч 44м			
5										

Рис. 16. Пример сгенерированного отчета

В отчете отражены ФИО всех сотрудников и статистика их рабочего времени с разделением на общее время, активное время и время использования приложений.

Таким образом подтверждается функционирование данного модуля, а также взаимодействие системы с базой данных.

4.3. Оценка эффективности системы

Применение системы учета и контроля рабочего времени сотрудников существенно повышает удобство в формировании и генерации отчетных документов, а также повышает эффективность управления персоналом путем отслеживания продуктивности в течение рабочего дня.

Благодаря возможности указывать перерывы, переработки командировки и больничные бухгалтерии больше не потребуется вручную формировать итоговую статистику отработанных часов по каждому сотруднику из различных источников (служебные записки, заявления и т.д.). В выгружаемом отчете будут учтены все факторы.

Без использования СКРВ бухгалтерия тратила около 2 минут на дополнительную проверку наличия у сотрудника командировок, больничных, переработок и около 3 минут на внесение этой информации в общие отработанные за месяц часы.

Использование СКРВ позволяет полностью исключить шаги по дополнительным проверкам, так как все данные уже учтены в автоматически сгенерированном отчете.

Для штата из 100 человек, в котором для порядка 40% сотрудников требуется учесть наличие вышеперечисленных факторов, влияющих на итоговое количество отработанных за месяц часов, а для остальных произвести проверку наличия этих факторов, общее количество затраченных минут T будет равно:

$$T = 2 \times 60 + 3 \times 40 = 120 + 120 = 240$$

Таким образом использование разработанного решения позволит уменьшить трудозатраты по расчету рабочего времени сотрудников на 4 часа, что является существенным результатом. Также стоит отметить, что уменьшается вероятность ошибок при формировании итоговых таблиц, так как объединение данных из разных источников вручную неизбежно приводит к ошибке, это человеческий фактор, а с помощью веб-приложения сотрудник сразу получает полные данные по всему персоналу.

С точки зрения проектных менеджеров и начальников отделов, использование разработанной системы позволит повысить эффективность управления сотрудниками, снизить риск необоснованной критики, так как в статистике можно будет посмотреть какие приложения использовал тот или иной человек и вовремя предпринимать меры по поощрению или взысканию.

ЗАКЛЮЧЕНИЕ

Выпускная квалификационная работа была посвящена разработке автоматизированной системы учета и контроля рабочего времени сотрудников, которая позволяет повысить эффективность управления персоналом, снизить трудозатраты и вероятность ошибки бухгалтерии при расчете фактически отработанных за месяц часов для формирования расчетных листов.

Анализ существующих решений показал, что продукты на рынке не отвечают в полной мере предъявляемым к системе требованиям безопасности, надежности и функциональности. Основным критерием отбора стала совместимость с отечественной операционной системой Astra Linux, применяемая в большом количестве предприятий нашей страны. Тем самым возникла идея разработки подобного приложения, которое сможет решить, поставленные перед ним задачи, учитывая вышеперечисленные требования.

Проект был реализован в несколько ключевых этапов:

1. Обзор предметной области. Были изучены основы построения систем контроля рабочего времени, изучены различные подходы к реализации.
2. Проведен анализ существующих решений: Yaware.TimeTracker, StaffCop, Мегаплан, Kickidler, CrocoTime. Выделены их преимущества и недостатки.
3. На основании результатов анализа были сформированы функциональные требования, требования к надежности и безопасности системы, а также требования к составу и параметрам технических средств.
4. Проектирование решения. Проведен сравнительный анализ монолитного и микросервисного подходов, в котором лучшим оказался второй вариант.
5. Разработана архитектура приложения. Для реализации клиентской части использовались: Vue.js, LocalStorage, Vue Router, Vuex, Axios. Для реализации серверной части использовались: FastAPI,

PostgreSQL, JWT, Uvicorn, SQLAlchemy. Сетевые технологии и инструменты разворачивания: RESTful API, HTTPS, WebSockets, CORS, Docker, Docker Compose.

6. Разработка. В соответствии с предъявляемыми требованиями, предложенной архитектурой и стеком технологий было реализовано приложение, состоящее из следующих основных компонентов: система аутентификации и авторизации, модуль учета рабочего времени, система мониторинга активности, модуль отчетности и аналитики, модуль администрирования системы, модуль управления учетной записью пользователя.
7. Тестирование. Проведен процесс разворачивания на тестовом стенде с проверкой работоспособности заявленной функциональности системы.

По результатам тестирования дана оценка эффективности разработанной системы. Решение позволило сократить трудозатраты по подсчету рабочего времени сотрудников на 4 часа, а также повысило эффективность управления персоналом путем возможности отслеживания активности в различных приложениях и просмотра подробной статистики по каждому сотруднику.

Все задачи, сформулированные во введении, выполнены.

К перспективам дальнейшего развития системы можно отнести интеграцию с системами контроля задач, ERP-платформами и CRM-системами, реализацию редактора отчетных документов прямо в интерфейсе приложения, улучшение алгоритмов анализа продуктивности сотрудников (возможно с использованием нейронных сетей), повышение уровня безопасности за счет многофакторной аутентификации и улучшение пользовательского опыта за счет активного взаимодействия с потребителями.

Данная работа подтверждает актуальность и перспективность использования веб-приложений в области управления персоналом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Docker Documentation [Электронный ресурс]. – URL: <https://docs.docker.com>
2. PostgreSQL Global Development Group. PostgreSQL 17.0 Documentation [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/17>
3. Ким Э. С. Анализ существующего программного обеспечения для автоматизации работы предприятия // Техника. Технологии. Инженерия. – 2019. – №1. – С. 11-14. – URL: <https://moluch.ru/th/8/archive/36/857/>
4. Шайбаков И. А. Принципы автоматизации рабочего времени сотрудников телекоммуникационной компании М: Уральский государственный экономический университет, 2020 10-15 с.
5. Модели данных и модели базы данных [Электронный ресурс]: – URL: https://studme.org/93786/informatika/modeli_dannyh_modeli_bazy_dannyh
6. Вигерс К. Разработка требований к программному обеспечению. 3-е изд., дополнительное / К. Вигерс, Д. Битти., Пер. с англ. – М.: Издательство «Русская редакция»; СПб.: БХВ-Петербург, 2014. – 736 с.
7. Система учета рабочего времени как элемент контроля эффективности работы сотрудников // Elibrary. — URL: <https://elibrary.ru/item.asp?id=48516470>
8. FastAPI Tiangolo. FastAPI Documentation [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com>
9. Vue.js Vue.js. Vue.js Official Guide [Электронный ресурс]. – URL: <https://vuejs.org>
10. SQLAlchemy SQLAlchemy. SQLAlchemy 2.0 Documentation [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org/en/20/>
11. RESTful API Fielding, R.T. Architectural Styles and the Design of Network-based Software Architectures (PhD thesis) [Электронный ресурс]. – URL: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

12. Методы автоматизации учета рабочего времени сотрудников предприятия // Elibrary. — URL: <https://www.elibrary.ru/item.asp?id=71489574>
13. Использование контейнеризации микросервисов для обеспечения безопасности веб-приложений // Elibrary. — URL: <https://elibrary.ru/item.asp?id=59887305>
14. Пьюривал С. Основы разработки веб-приложений / Пьюривал С.: Издательство «Питер», 2015. – 272 с.
15. Коберн А. Современные методы описания функциональных требований к системам. – М.: издательство «Лори», 2012. – 263 с.
16. Тестирование безопасности как обязательный этап разработки веб-приложения // Elibrary. — URL: <https://elibrary.ru/item.asp?id=68607635>